WINDOWS · GIT · SSH

Using Multiple GitHub Accounts on One Computer with SSH

What was set up for **Mohanvamsi4405**, how to do the whole thing again from scratch, and how to add any new GitHub account later.

Contents

1. What I did (Mohanvamsi4405)
2. How it works
3. The values you change
4. From scratch: full setup
5. Adding another account
6. Using an account in a repo
7. My accounts right now
8. Troubleshooting
9. Cheat sheet

1 What I did (Mohanvamsi4405)

These are the exact commands I ran to connect the new account **Mohanvamsi4405** to the repo in `D:\GIT`, while keeping my existing **theaishastra** account working as before.

1 Generated a new SSH key just for this account

```
ssh-keygen -t ed25519 -C "chitradamohanvamsi33@gmail.com" -f
"%USERPROFILE%\.ssh\id_ed25519_mohan_github_33"
```

This made two files in `C:\Users\mohan\.ssh\` : `id_ed25519_mohan_github_33` (private, never share it) and `id_ed25519_mohan_github_33.pub` (public, this one goes to GitHub).

2 Printed the public key and copied it

```
type "%USERPROFILE%\.ssh\id_ed25519_mohan_github_33.pub"
```

3

Added the key to GitHub (signed in as Mohanvamsi4405)

GitHub → profile picture → **Settings** → **SSH and GPG keys** → **New SSH key** → Title: *Windows Laptop* → Key type: **Authentication Key** → paste → **Add SSH key**.

4

Added an alias to the SSH config file

```
notepad "%USERPROFILE%\.ssh\config"
```

Then added this block and saved:

```
# Main Mohan 33 work repo
Host github-mohan33
    HostName github.com
    User git
    IdentityFile ~/.ssh/id_ed25519_mohan_github_33
    IdentitiesOnly yes
```

5

Tested the connection

```
ssh -T git@github-mohan33
```

Result: `Hi Mohanvamsi4405! You've successfully authenticated…` ✅

6

Pointed the repo at the new alias

```
D:
cd \GIT
git remote set-url origin git@github-
mohan33:Mohanvamsi4405/GIT_HUB_LEARNING.git
git remote -v
```

7

Pushed the code

```
git push -u origin main
```

⚠ One thing still to fix

The push now goes through **Mohanvamsi4405**'s key, but this repo's commit *author* is still set to `theaishastra` / `theaishastra@gmail.com` (from the global Git config). On GitHub, the commits will show up under **theaishastra**. To fix it for this repo only:

```
cd /d D:\GIT
git config user.name "Mohanvamsi4405"
git config user.email "chitradamohanvamsi33@gmail.com"
```

This changes future commits only. It doesn't rewrite the ones you already made.

2 How it works

GitHub works out which account you are from the **SSH key** you connect with. An alias (a fake hostname) tells SSH which key to use.

REPO REMOTE URL
`git@github-mohan33:Mohanvamsi4405/REPO.git`

→

~/.SSH/CONFIG
`Host github-mohan33`
→ HostName github.com
→ use key `id_ed25519_mohan_github_33`

→

GITHUB
Matches the key to the account it was added to
→ **Mohanvamsi4405**

The key idea

`git@github.com:…` uses your **default** key (theaishastra).
`git@github-mohan33:…` uses the **Mohanvamsi4405** key.
The only difference is the hostname in the URL. The SSH config file does the rest.

`IdentitiesOnly yes` is important. It makes SSH use *only* the key you named, so it can't log in with another key by mistake.

3 The values you change

In the commands below, anything shown like `<THIS>` is a **value you replace**. Everything else is typed exactly as shown.

Placeholder	What it means	Example (what I used)
`<EMAIL>`	Email of the GitHub account (just a label on the key)	`chitradamohanvamsi33@gmail.com`
`<KEY_NAME>`	File name for the new key. Must be **unique**, so you don't overwrite another key	`id_ed25519_mohan_github_33`
`<ALIAS>`	Made-up hostname for this account. Any name works, but no spaces	`github-mohan33`
`<GH_USER>`	GitHub username that owns the repo	`Mohanvamsi4405`
`<REPO>`	Repository name on GitHub (create it on GitHub first)	`GIT_HUB_LEARNING`
`<FOLDER>`	Local project folder	`D:\GIT`
`<NAME>`	Name shown on commits	`Mohanvamsi4405`

How often each step is done

`PER ACCOUNT` once for each GitHub account `PER REPO` once for each project folder

> **CMD or PowerShell?**
> The commands here are for **Command Prompt (CMD)**. In **PowerShell**, replace `%USERPROFILE%` with `$env:USERPROFILE` and `type` with `Get-Content`.

4 From scratch: full setup

Use this when setting up an account on a new computer, or when starting over from nothing.

0 Check what keys already exist PER ACCOUNT

```
dir "%USERPROFILE%\.ssh"
```

Pick a `<KEY_NAME>` that isn't already in the list. If the folder doesn't exist, the next step creates it.

1 Generate the key PER ACCOUNT

```
ssh-keygen -t ed25519 -C "<EMAIL>" -f "%USERPROFILE%\.ssh\<KEY_NAME>"
```

Passphrase: press Enter to skip it, or type one for extra safety.

2 Copy the public key PER ACCOUNT

```
type "%USERPROFILE%\.ssh\<KEY_NAME>.pub"
```

Shortcut that copies it straight to the clipboard:

```
type "%USERPROFILE%\.ssh\<KEY_NAME>.pub" | clip
```

> **Only the .pub file**
> Copy only the file that ends in `.pub`. The file without `.pub` is your private key. Never paste it anywhere.

3 Add it on GitHub PER ACCOUNT

Sign in as `<GH_USER>` (an incognito window makes this easier) → **Settings** → **SSH and GPG keys** → **New SSH key** → type **Authentication Key** → paste → **Add SSH key**.

Direct link: `https://github.com/settings/keys`

4 Add the alias to the SSH config PER ACCOUNT

```
notepad "%USERPROFILE%\.ssh\config"
```

Add this at the bottom (one block per account) and save:

```
Host <ALIAS>
    HostName github.com
    User git
    IdentityFile ~/.ssh/<KEY_NAME>
    IdentitiesOnly yes
```

> **Notepad trap**
>
> The file must be named exactly `config`, with **no** `.txt`. If Notepad asks to create a new file, click Yes. When saving, pick "All files" so it doesn't become `config.txt`.

5 Test the connection PER ACCOUNT

```
ssh -T git@<ALIAS>
```

Expected: `Hi <GH_USER>! You've successfully authenticated…`
The first time, type `yes` to trust github.com.

6 Connect a project. Choose **one** option PER REPO

Option A: a new local project you want to upload

```
cd /d <FOLDER>
git init
git config user.name "<NAME>"
git config user.email "<EMAIL>"
git add .
git commit -m "First commit"
git branch -M main
git remote add origin git@<ALIAS>:<GH_USER>/<REPO>.git
git push -u origin main
```

Option B: an existing repo that already has a remote (this is what I did)

```
cd /d <FOLDER>
git remote set-url origin git@<ALIAS>:<GH_USER>/<REPO>.git
git config user.name "<NAME>"
git config user.email "<EMAIL>"
git push -u origin main
```

Option C: clone a repo from GitHub

```
git clone git@<ALIAS>:<GH_USER>/<REPO>.git
cd <REPO>
git config user.name "<NAME>"
git config user.email "<EMAIL>"
```

> **Tip**
>
> When GitHub shows a clone URL like `git@github.com:User/Repo.git`, replace `github.com` with your `<ALIAS>`.

7 Check it PER REPO

```
git remote -v
git config user.name
git config user.email
```

5 Adding another account

Your computer already has an SSH config and other keys. To add one more account (for example, a 5th one), repeat **steps 1–5** with new values. Don't touch the existing blocks.

Worked example: adding a made-up account `newuser99`

```
:: 1. New key (unique file name!)
ssh-keygen -t ed25519 -C "newuser99@gmail.com" -f
"%USERPROFILE%\.ssh\id_ed25519_newuser99"

:: 2. Copy public key to the clipboard
type "%USERPROFILE%\.ssh\id_ed25519_newuser99.pub" | clip

:: 3. Paste it at https://github.com/settings/keys  (signed in as
newuser99)

:: 4. Append the alias to the config
notepad "%USERPROFILE%\.ssh\config"
```

```
# newuser99 account
Host github-newuser99
    HostName github.com
    User git
    IdentityFile ~/.ssh/id_ed25519_newuser99
    IdentitiesOnly yes
```

```
:: 5. Test
ssh -T git@github-newuser99

:: 6. Use it in a repo
git remote set-url origin git@github-newuser99:newuser99/my-repo.git
```

Checklist

- ✓ The new key file name doesn't match any existing key
- ✓ The public key (`.pub`) is added to the **correct** GitHub account
- ✓ The new `Host` alias is unique in the config file
- ✓ `ssh -T git@<ALIAS>` greets the right username
- ✓ The repo remote uses `git@<ALIAS>:`, not `git@github.com:`

✓ The repo's `user.name` and `user.email` are set to that account

> **A key can belong to only one account**
> GitHub won't let you add the same public key to two accounts ("Key is already in use"). Each account needs its own key.

6 Using an account in a repo

Which account a repo uses depends on two settings, and both need to match the account:

Setting	Controls	Command
Remote URL	Which account **pushes** (needs write access)	`git remote set-url origin git@<ALIAS>:<GH_USER>/<REPO>.git`
user.name / user.email	Who appears as the **commit author**	`git config user.email "<EMAIL>"`

Without `--global`, `git config` applies to the current repo only, which is what you want when using several accounts.

7 My accounts right now

From `C:\Users\mohan\.ssh\config`. Use the alias in the URL column to pick an account.

Account	Alias (Host)	Key file	Remote URL starts with
theaishastra (default)	`github.com`	`id_ed25519`	`git@github.cc`
MohanVamsi4455 (Jobs)	`github-mohanvamsi4455`	`id_ed25519_mohanvamsi4455`	`git@github-mohanvamsi445`
Wellytics	`github-wellytics`	`id_ed25519_wellytics`	`git@github-wellytics:`
Mohanvamsi4405 (new)	`github-mohan33`	`id_ed25519_mohan_github_33`	`git@github-mohan33:`

8 Troubleshooting

Problem	Cause → fix
`Permission denied (publickey)`	The key isn't on GitHub, or the path in the config is wrong. Check the `IdentityFile` path, then run `ssh -vT git@<ALIAS>` to see which key is tried.
`Hi theaishastra!` instead of the new account	The remote still uses `github.com`, or `IdentitiesOnly yes` is missing. Fix the remote URL and the config block.
`ERROR: Repository not found`	The repo doesn't exist on GitHub yet, the name is misspelled, or this account has no access. Create the repo, or add the account as a collaborator.
`Could not resolve hostname github-xxx`	The alias isn't in the config, or the file is saved as `config.txt`. Check with `dir "%USERPROFILE%\.ssh"`.
`Key is already in use` on GitHub	That key is already on another account. Make a new key for this account.
Commits show the wrong person on GitHub	The repo's `user.email` belongs to another account. Set it with `git config user.email "<EMAIL>"`.
`failed to push… rejected (fetch first)`	The GitHub repo has commits you don't have (for example, a README). Run `git pull origin main --rebase`, then push.

Useful check commands

```
:: Which account does an alias log in as?
ssh -T git@<ALIAS>

:: Verbose output (shows which key file is used)
ssh -vT git@<ALIAS>

:: Show the SSH config / keys
type "%USERPROFILE%\.ssh\config"
dir "%USERPROFILE%\.ssh"

:: Show the current repo's remote and author
git remote -v
git config user.name & git config user.email
```

9 Cheat sheet

The whole process on one card. Replace the `<highlighted>` values.

```
:: ── Per account (one time) ──────────────────────────
ssh-keygen -t ed25519 -C "<EMAIL>" -f "%USERPROFILE%\.ssh\<KEY_NAME>"
type "%USERPROFILE%\.ssh\<KEY_NAME>.pub" | clip       :: paste at
github.com/settings/keys
notepad "%USERPROFILE%\.ssh\config"                 :: add Host block
(below)
ssh -T git@<ALIAS>                                :: expect "Hi
<GH_USER>!"

:: ── Per repo ────────────────────────────────────────
cd /d <FOLDER>
git remote set-url origin git@<ALIAS>:<GH_USER>/<REPO>.git   :: or
"remote add" if new
git config user.name  "<NAME>"
git config user.email "<EMAIL>"
git push -u origin main
```

```
# ~/.ssh/config block
Host <ALIAS>
    HostName github.com
    User git
    IdentityFile ~/.ssh/<KEY_NAME>
    IdentitiesOnly yes
```